

Fundamentos de Programación

Algoritmos y Programación I

Semana 7 — Clase Práctica en Laboratorio**Punteros, Referencias, Modelo de Memoria y Primeros Algoritmos Recursivos****Introducción**

Los problemas de esta semana exploran el **modelo de memoria de Python**: cómo las variables referencian objetos en el heap, qué consecuencias tiene la **mutabilidad** sobre el *aliasing* y el paso de argumentos a funciones, y cómo la **pila de ejecución** gestiona las invocaciones de funciones — incluyendo los primeros algoritmos **recursivos**. El eje temático es la comprensión profunda de *por qué* ciertos comportamientos de Python que parecen misteriosos (modificar una lista dentro de una función afecta al exterior; reasignar un entero no) son consecuencias directas del modelo de referencias.

Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** (Análisis → Diseño → Codificación → Evaluación) y el **modelo de programa modular** de la Semana 4: cada función incluye su sección declarativa (docstring con precondition, postcondition y, cuando corresponda, **efectos secundarios**).

Los seis problemas están organizados en tres bloques con **conexiones deliberadas**: los Problemas 1 y 2 exploran identidad, *aliasing* y mutabilidad como laboratorio de descubrimiento; los Problemas 3 y 4 formalizan el paso de argumentos y los efectos secundarios conectando con las funciones de ordenamiento de la Semana 6; y los Problemas 5 y 6 introducen la recursión vinculándola con la pila de ejecución estudiada en la Semana 4 y profundizada en la clase teórica de esta semana.

Recordatorio: Modelo de programa modular — Cada función incluye su sección declarativa como **docstring** (descripción, precondition, postcondition, efectos secundarios si los hay), accesible mediante `help()`. La sección algorítmica se organiza en Prólogo / Resolución / Epílogo.

Herramienta clave: Python Tutor — En esta práctica se utilizará **Python Tutor** (pythontutor.com) para visualizar paso a paso la creación de objetos en el heap, las referencias desde el stack, y los efectos del *aliasing*. Verificar cada predicción ejecutando el código en Python Tutor **antes** de ejecutarlo en el intérprete.

Conexión con la Semana 4 — El **simulador interactivo de pila de ejecución** del documento 'Funciones, Modularización y Abstracción' (Sección 6.2) es el complemento directo de los Problemas 5 y 6. Repasar ese simulador antes de abordar la recursión ayuda a consolidar el modelo mental de apilar/desapilar frames.

Bloque A: Identidad, referencias y aliasing

Problema 1: Exploración de identidad y aliasing con Python Tutor

Enunciado

Escribir un programa en Python que realice los siguientes experimentos, registrando las predicciones *antes* de ejecutar cada bloque y verificándolas después con `id()` y con Python Tutor:

Experimento 1 — Inmutables:

(a) Asignar `x = 10` y `y = x`. Predecir: ¿`x` is `y`? Verificar con `id()`. Luego ejecutar `y = y + 1`. Predecir: ¿`x` is `y` ahora? ¿Cambió `x`? Verificar.

(b) Repetir con cadenas: `s1 = "hola"` y `s2 = s1`. Ejecutar `s2 = s2 + " mundo"`. Predecir y verificar.

(c) Asignar `a = 256` y `b = 256`. Verificar `a is b`. Luego `a = 257` y `b = 257`. Verificar `a is b`. Explicar la diferencia (caché de enteros de CPython: rango [-5, 256]).

Experimento 2 — Mutables:

(d) Asignar `lista1 = [1, 2, 3]` y `lista2 = lista1`. Ejecutar `lista2.append(4)`. Predecir: ¿cambió `lista1`? Verificar con `id()` e `is`.

(e) Asignar `lista3 = [1, 2, 3]` y `lista4 = [1, 2, 3]`. Verificar: ¿`lista3 == lista4`? ¿`lista3 is lista4`? Explicar la diferencia.

(f) Asignar `tupla1 = (10, 20)` y `tupla2 = tupla1`. Intentar `tupla2[0] = 99`. Registrar el error y explicar por qué las tuplas son inmutables.

Experimento 3 — Aliasing en listas anidadas:

(g) Crear `matriz = [[1, 2], [3, 4]]`. Hacer `copia = matriz[:]`. Modificar `copia[0][0] = 99`. Predecir: ¿cambió `matriz`? Verificar. Explicar por qué la copia superficial no protege las sublistas.

(h) Repetir (g) usando `import copy; copia_profunda = copy.deepcopy(matriz)`. Verificar que la modificación de `copia_profunda` no afecta a `matriz`.

Orientaciones para la resolución

Análisis: Antes de cada experimento, anotar en un comentario la predicción (qué valores tendrán las variables, qué retornará `is`). Esto transforma el ejercicio de “ejecutar y ver” en “predecir, ejecutar y comparar”.

Diseño: Organizar cada experimento en una función separada que imprima los resultados con formato claro, incluyendo `id()` de cada variable.

Evaluación: Comparar la salida del intérprete con la visualización en Python Tutor. Identificar dónde las predicciones fueron incorrectas y por qué.

Conexión con la clase teórica — Este problema ejercita directamente los conceptos de las Secciones 2, 3 y 4 del documento teórico: el modelo de cuatro segmentos (*stack* y *heap*), la diferencia entre `==` e `is`, y la clasificación de tipos en *mutables* e *inmutables*. La copia superficial vs. profunda anticipa situaciones que reaparecerán en la Semana 10 al trabajar con listas de listas (*matrices*).

Problema 2: Aliasing en funciones — detectar, documentar y prevenir

Enunciado

Analizar el siguiente programa *sin ejecutarlo*, predecir la salida, y luego verificar con Python Tutor:

```
def agregar_elemento(lista, elemento):
    """Agrega un elemento al final de la lista.

    Precondición: lista es una lista, elemento es cualquier valor.
    Postcondición: la lista queda con el elemento agregado al final.
    Efecto secundario: modifica la lista recibida.
    """
    lista.append(elemento)
    return lista

def intentar_reemplazar(lista, nueva):
    """Intenta reemplazar la lista por otra.

    Precondición: lista y nueva son listas.
    Postcondición: retorna nueva; la lista original NO se modifica.
    """
    lista = nueva
    return lista

# === Sección algorítmica ===
# --- Prólogo ---
original = [10, 20, 30]
print(f"Antes:      original = {original}, id = {id(original)}")

# --- Resolución ---
resultado1 = agregar_elemento(original, 40)
print(f"Después 1: original = {original}, id = {id(original)}")
print(f"              resultado1 is original: {resultado1 is original}")

resultado2 = intentar_reemplazar(original, [99, 98, 97])
print(f"Después 2: original = {original}, id = {id(original)}")
print(f"              resultado2 = {resultado2}")
print(f"              resultado2 is original: {resultado2 is original}")
```

A continuación:

- (a) Explicar en un comentario de varias líneas por qué `agregar_elemento` modifica la lista original pero `intentar_reemplazar` no.
- (b) Escribir una versión **sin efecto secundario** de `agregar_elemento` llamada `agregar_elemento_puro(lista, elemento)` que retorne una nueva lista sin modificar la original. Documentar la ausencia de efecto secundario en la docstring.
- (c) Escribir una función `extender_si_corta(datos, minimo, valor_relleno)` que, si la lista `datos` tiene menos de `minimo` elementos, la extienda hasta alcanzar la longitud `minimo`. Decidir y documentar si la función tiene o no efecto secundario, y justificar la decisión.

Orientaciones para la resolución

Análisis: La clave es distinguir *modificación del objeto* (`append`, que opera sobre el mismo `id`) de *reasignación del parámetro* (`lista = nueva`, que solo reapunta el nombre local).

Diseño: En (b), la forma más directa es `return lista + [elemento]`. En (c), si se elige efecto secundario, usar un `while` que agregue elementos hasta alcanzar la longitud (bucle indeterminado → `while`, no `for`).

Conexión con la Semana 6 — Ahora se puede entender por qué `lista.sort()` ordena *in-place* (modifica el objeto, efecto secundario) mientras que `sorted(lista)` retorna una nueva lista (sin efecto secundario). El Problema 3 retoma esta distinción con un ejemplo concreto.

Bloque B: Paso de argumentos y conexión con ordenamiento

Problema 3: Ordenamiento con y sin efectos secundarios — análisis con `id()`

Enunciado

Retomar la función `ordenamiento_seleccion(datos)` de la Semana 6 y la función `ordenar_sin_modificar(datos)` de la Sección 6.3 del documento teórico de esta semana. Escribir un programa que:

- (a) Genere una lista de 10 enteros pseudoaleatorios en `[1, 50]` con `random.seed(42)`.
- (b) Invoque `ordenamiento_seleccion` sobre una copia de la lista. Imprimir el `id()` antes y después. Verificar que el `id` no cambia (mismo objeto, modificado *in-place*).
- (c) Invoque `ordenar_sin_modificar` pasando la lista original. Imprimir el `id()` de la lista original y del resultado. Verificar que los `id` son diferentes y que la lista original no se modificó.
- (d) Comparar con el comportamiento de `sorted()` (built-in) y `.sort()` (método de lista).
- (e) Escribir una tabla comparativa como comentario multilínea al final del programa.

Orientaciones para la resolución

Análisis: Este problema cierra el ciclo que la Semana 6 dejó abierto: en aquella semana implementamos los algoritmos de ordenamiento sin comprender el mecanismo por el cual un sort in-place modifica la lista del llamador. Ahora, con el modelo de referencias, la explicación es precisa.

Diseño: Usar la estructura modular con un bloque de prueba por cada variante. Separar claramente la creación de copias (`datos.copy()` o `datos[:]`) para evitar que una prueba contamine la siguiente.

Problema 4: Análisis empírico de uso de memoria con `sys.getsizeof()`

Enunciado

Escribir un programa modular que investigue cuánta memoria ocupan los distintos tipos de objetos en Python:

- (a) Medir y comparar el tamaño en bytes de: un int pequeño (42), un int grande (2^{**100}), un float (3.14), un bool (True), una cadena corta ("hola") y una cadena larga ("a" * 1000).
- (b) Medir el tamaño de listas de enteros para $n \in \{0, 1, 10, 100, 1000\}$. Calcular el overhead de la estructura lista y el tamaño total incluyendo los objetos referenciados. Presentar en tabla formateada.
- (c) Comparar el tamaño de una list vs. una tuple con los mismos 100 elementos. ¿Cuánto ahorra la tupla? ¿Por qué?
- (d) Medir el efecto de la sobreasignación de CPython: comparar el tamaño de una lista creada con `[1, 2, 3] + [4]` vs. una lista a la que se le hizo `append(4)`. Explicar la diferencia.

Orientaciones para la resolución

Análisis: `sys.getsizeof(obj)` devuelve el tamaño directo del objeto en bytes, pero para contenedores no incluye el tamaño de los objetos referenciados. Para el tamaño total:
`total = sys.getsizeof(lista) + sum(sys.getsizeof(e) for e in lista).`

Diseño: Definir una función `medir_tamano(objeto, nombre)` que imprima nombre, tipo y tamaño en una línea formateada.

Conexión holística: la jerarquía de memoria — Cada byte que ocupa un objeto en el heap es un byte de RAM que no está disponible para otros procesos. La Sección 2 del documento 'Funciones del Sistema Operativo' explica cómo el SO gestiona esta RAM mediante paginación: las páginas de 4 KB que contienen nuestros objetos Python se mapean a marcos físicos, y cuando la RAM se agota, el SO puede enviar páginas al disco (swapping).

Bloque C: Primeros algoritmos recursivos y pila de ejecución

Problema 5: Factorial, suma de lista y potencia — recursión y traza de la pila

Enunciado

Implementar tres funciones recursivas clásicas y analizar su comportamiento en la pila de ejecución:

- (a) `factorial(n)`: retorna $n!$ para $n \geq 0$ (caso base: $0! = 1$).
- (b) `suma_lista(datos)`: retorna la suma de los elementos de una lista (caso base: lista vacía $\rightarrow 0$).
- (c) `potencia(base, exponente)`: retorna $\text{base}^{\text{exponente}}$ para $\text{exponente} \geq 0$ (caso base: $\text{exponente} == 0 \rightarrow 1$).

Para cada función:

1. Documentar el caso base y el caso recursivo en la docstring.
2. Agregar una instrucción `print` al inicio de cada invocación que muestre el nivel de profundidad y los argumentos recibidos.
3. Verificar la correctitud con al menos tres casos de prueba incluyendo el caso base.
4. Ejecutar `factorial(4)` y `suma_lista([3, 7, 2, 5])` en Python Tutor y observar la pila de frames.
5. Trazar manualmente la pila de ejecución de `potencia(2, 4)` en una tabla.

Ejemplo de salida esperada para `factorial(4)`:

```
factorial(4) - profundidad 1
  factorial(3) - profundidad 2
    factorial(2) - profundidad 3
      factorial(1) - profundidad 4
        factorial(0) - profundidad 5 [caso base]
```

Orientaciones para la resolución

Análisis: Cada invocación recursiva apila un nuevo frame en el stack. Para `factorial(n)`, la profundidad máxima es $n + 1$ frames (incluyendo el caso base).

Diseño: Usar un parámetro auxiliar `profundidad=1` con valor por defecto para el rastreo. La indentación del `print` se logra con `" " * profundidad`.

Evaluación: Verificar que `factorial(0) == 1`, `factorial(5) == 120`, `suma_lista([]) == 0`, `potencia(2, 10) == 1024`.

Conexión con la pila de ejecución (Semana 4) — En la Sección 6 del documento ‘Funciones, Modularización y Abstracción’ estudiamos que cada invocación de función apila un stack frame con parámetros, variables locales y dirección de retorno. La recursión es la aplicación repetida de este mecanismo. El **simulador interactivo** de esa

sección permite observar el apilamiento paso a paso — reutilizarlo con las funciones recursivas de este problema.

Problema 6: Profundidad de recursión, RecursionError y conexión con el stack del SO

Enunciado

Investigar experimentalmente los límites de la recursión en Python y su relación con la pila del sistema operativo:

- (a) Escribir una función `cuenta_regresiva(n)` que imprima $n, n-1, \dots, 1, 0$ recursivamente (caso base: $n < 0$).
- (b) Invocarla con $n = 10$, $n = 100$, $n = 500$. ¿Funciona correctamente?
- (c) Consultar el límite actual de recursión con `sys.getrecursionlimit()`. Invocar `cuenta_regresiva` con un valor superior. Registrar el `RecursionError`.
- (d) Modificar temporalmente el límite con `sys.setrecursionlimit(50)`. Invocar `cuenta_regresiva(60)`. Restaurar el límite original.
- (e) Escribir una versión **iterativa** `cuenta_regresiva_iter(n)` que produzca la misma salida. Verificar que funciona para $n = 10000$ sin error.
- (f) Responder en un comentario multilínea: ¿Por qué CPython impone un límite de recursión? ¿Qué sucedería en la pila del SO si no existiera? ¿Qué relación tiene con el stack overflow descrito en la Sección 8 del documento teórico y en la Sección 2 del documento 'Funciones del Sistema Operativo'?

Orientaciones para la resolución

Análisis: Python permite ~1000 niveles de recursión por defecto. Sin este límite, una recursión infinita consumiría todo el stack hasta producir un stack overflow a nivel del SO.

Diseño: Usar `try / except RecursionError as e:` para capturar el error sin que el programa termine. La versión iterativa usa un `while` con decremento.

Conexión holística: el stack del SO — La Sección 2 del documento 'Funciones del Sistema Operativo' describe cómo el SO asigna un stack de tamaño fijo a cada proceso. Un `RecursionError` de Python es una protección **antes** de que el stack del SO se agote. Sin esta protección, el proceso recibiría una señal `SIGSEGV` del SO y terminaría abruptamente.

Distribución sugerida del tiempo (9 horas)

Actividad	Problemas / Tareas	Tiempo	Contexto
Clase práctica (3 h)	Problemas 1 y 2	~1.5 h	Exploración guiada con Python Tutor

	Problema 3	~45 min	Conexión con ordenamiento de Semana 6
	Problema 5 (inicio)	~45 min	Primeras funciones recursivas con traza
Extracursado (6 h)	Relectura: Gestión de Memoria en 'Funciones del SO' (Secc. 2)	~45 min	Memoria virtual, paginación, segmentación
	Relectura: Pila de ejecución y simulador en 'Funciones' (Secc. 6)	~30 min	Revisitar el simulador con la perspectiva de la recursión
	Problema 4 (sys.getsizeof)	~1.5 h	Análisis empírico de uso de memoria
	Problemas 5 (fin) y 6	~2 h	Recursión, RecursionError y stack del SO
	Foro: paso por valor vs. referencia	~1 h	Comparar Python, C, Java
	Autoevaluación (Moodle)	~15 min	Verificación individual

Criterio de distribución — Los Problemas 1 y 2 requieren descubrimiento guiado con Python Tutor y se benefician de la asistencia presencial. El Problema 3 conecta con la Semana 6 y es breve porque reutiliza código existente. El Problema 5 se inicia en clase para que el docente oriente la primera traza recursiva. Las lecturas de extracursado son relecturas de secciones ya introducidas, ahora con la perspectiva ampliada de esta semana; se ubican al inicio del extracursado para que fundamenten el Problema 6. El foro requiere investigación sobre C y Java y se estima en 1 hora incluyendo redacción y lectura de aportes de compañeros.

Notas finales

Estos seis problemas cubren los contenidos prácticos de la **Semana 7** del curso. Los Problemas 1 y 2 construyen el modelo mental de referencias, aliasing y mutabilidad mediante exploración experimental con Python Tutor. El Problema 3 conecta ese modelo con las funciones de ordenamiento de la Semana 6. El Problema 4 vincula el modelo de memoria teórico con mediciones empíricas usando `sys.getsizeof()`. Los Problemas 5 y 6 introducen la recursión como caso particular de la pila de ejecución, estableciendo las bases para la formalización completa en la Semana 9.

Criterios de evaluación formativa

Además de los criterios de semanas anteriores (metodología de Pólya, modelo de programa, *literate programming*, contratos de funciones), esta semana se evaluarán especialmente:

Predicción antes de ejecución: cada experimento incluye una predicción escrita (como comentario) que luego se contrasta con el resultado real.

Uso correcto de `id()` e `is`: distinguir identidad de igualdad; usar `is` exclusivamente para comparación con `None` en código de producción.

Documentación de efectos secundarios: las funciones que modifican argumentos mutables declaran explícitamente el efecto secundario en la docstring.

Traza manual de la pila: las tablas de traza de recursión reflejan correctamente el apilamiento y desapilamiento de frames.

Correctitud de las funciones recursivas: caso base correcto, caso recursivo que avanza hacia el caso base, y verificación con al menos tres casos de prueba.

Para profundizar

(1) En el Problema 1, investigar: ¿Python realiza *string interning*? Verificar con `s1 = "hola"` y `s2 = "hola"` (ambas como literales): ¿`s1 is s2`? ¿Y si `s2` se construye dinámicamente como `"ho" + "la"`? (2) En el Problema 4, comparar el tamaño de un dict con 100 pares clave-valor vs. una lista de 100 tuplas (clave, valor). Esta diferencia anticipa el estudio de tablas hash en la Semana 10. (3) En el Problema 5, implementar `fibonacci(n)` recursivo y contar cuántas invocaciones se realizan para `n = 10, 20, 30`. ¿Por qué crece exponencialmente? Esta ineficiencia motiva la memoización (Semana 9). (4) En el Problema 6, investigar qué ocurre en C cuando una función recursiva se invoca sin caso base: ¿existe un `RecursionError` equivalente, o el programa termina con una señal del SO?

Guía para el Foro

Paso por valor y por referencia en distintos lenguajes

El foro propone comparar cómo tres lenguajes representativos manejan el paso de argumentos a funciones:

Python: paso por asignación (*pass by object reference*). Se copia la referencia, no el objeto. Modificar un argumento mutable es visible fuera; reasignar el parámetro no.

C: paso por valor estricto. Se copia el valor del argumento (incluido el valor de un puntero). Para modificar una variable del llamador se debe pasar explícitamente un puntero a ella.

Java: paso por valor de la referencia (similar a Python para objetos). Los tipos primitivos (`int`, `double`) se copian; los objetos se pasan como copia de la referencia.

Consigna: Presentar un ejemplo breve de código en al menos dos lenguajes (Python + uno más) que muestre la diferencia observable. Explicar qué predice cada modelo sobre el resultado. Leer al menos dos contribuciones de compañeros y comentar si el ejemplo presentado es consistente con el modelo descrito.